

FitBuddy: App.py Development Guide

AI Fitness Plan Generator using Gemini Models | Epic 3 Implementation Document

1. Overview of App.py & Architecture

In the *FitBuddy* architecture, *app.py* serves as the primary entry point for the FastAPI web application. It is responsible for initializing the FastAPI application instance, configuring middleware, registering route handlers (*routes.py*), managing database connections or session states, and integrating the Google Gemini generative AI SDK.

Key Milestone Objective: Establish a robust backend runner that boots up the server, wires up Jinja2 templates for the frontend, and securely handles API keys for the Gemini models without exposing them client-side.

2. Core Responsibilities of app.py

- **Application Initialization:** Instantiating *app = FastAPI(title="FitBuddy")* with appropriate metadata.
- **Static Files & Templates Configuration:** Mounting static directories (CSS, JS, images) and configuring Jinja2 templates for dynamic HTML rendering.
- **Router Inclusion:** Importing and including modular route handlers defined in *routes.py*.
- **Environment & Security Setup:** Loading environment variables (*python-dotenv*) to securely read the Gemini API key.

3. Reference Implementation Code

Below is a standard, production-ready structure for *app.py* tailored for the *FitBuddy* project:

```

import os
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from dotenv import load_dotenv
import google.generativeai as genai

# Load environment variables from .env file
load_dotenv()

# Configure Google Gemini API Key
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
if GEMINI_API_KEY:
    genai.configure(api_key=GEMINI_API_KEY)

# Initialize FastAPI application
app = FastAPI(
    title="FitBuddy - AI Fitness Plan Generator",
    description="Generates personalized workout and nutrition plans using Gemini Models.",
    version="1.0.0"
)

# Mount static files directory (CSS, JS, assets)
app.mount("/static", StaticFiles(directory="static"), name="static")

# Configure Jinja2 templates directory
templates = Jinja2Templates(directory="templates")

# Import and include routers from routes.py
from routes import router as main_router
app.include_router(main_router)

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("app:app", host="127.0.0.1", port=8000, reload=True)

```

4. Integration with routes.py

To keep the codebase modular, [app.py](#) delegates specific endpoint logic to [routes.py](#). The router shares template rendering capabilities and Gemini model invocation helper functions:

```
# Snippet showing how routes.py hooks into app templates
from fastapi import APIRouter, Request, Form
from fastapi.templating import Jinja2Templates

router = APIRouter()
templates = Jinja2Templates(directory="templates")

@router.get("/")
def read_index(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})
```

5. Verification & Local Execution

Once `app.py` and `routes.py` are properly configured alongside your Jinja2 HTML templates, you can launch the local development server using Uvicorn:

```
uvicorn app:app --reload
```

Open your browser and navigate to <http://127.0.0.1:8000> to interact with your newly initialized *FitBuddy* interface powered by Gemini models.